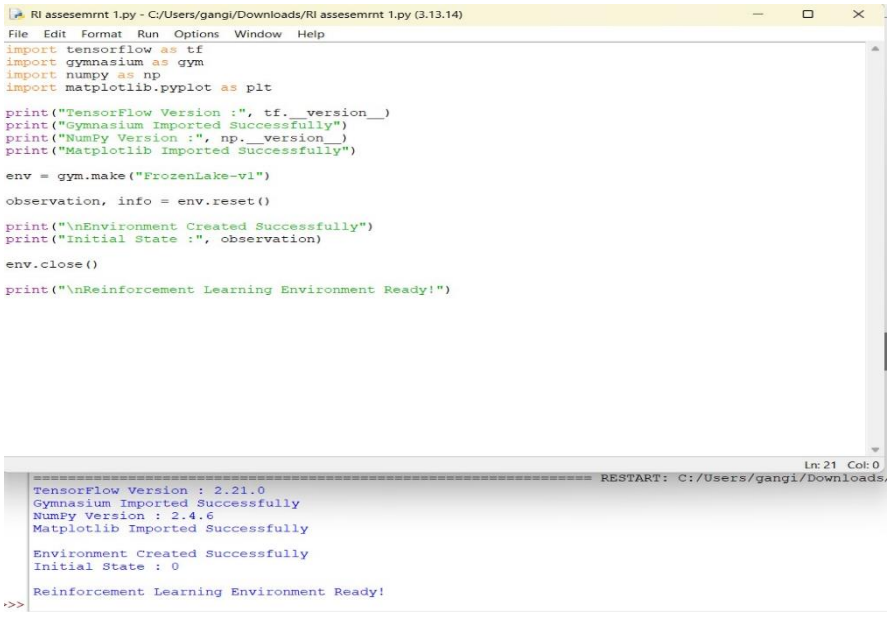
Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program

Program:

```
import tensorflow as tf

import gymnasium as gym

import numpy as np

import matplotlib.pyplot as plt

print("TensorFlow Version :", tf._version_)

print("Gymnasium Imported Successfully")

print("NumPy Version :", np._version_)

print("Matplotlib Imported Successfully")

env = gym.make("FrozenLake-v1")

observation, info = env.reset()

print("\nEnvironment Created Successfully")

print("Initial State :", observation)

env.close()

print("\nReinforcement Learning Environment Ready!")
```

Output:



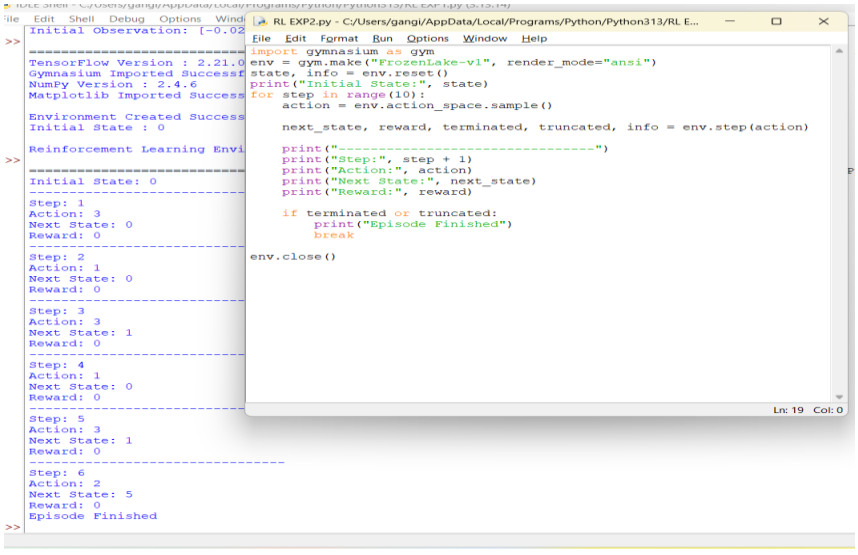
Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

AIM:

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and Mountain Car using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

PROGRAM:

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", render_mode="ansi")

state, info = env.reset()

print("Initial State:", state)

for step in range(10):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("--------------------------------")

    print("Step:", step + 1)

    print("Action:", action)

    print("Next State:", next_state)

    print("Reward:", reward)

    if terminated or truncated:

        print("Episode Finished")

        break

env.close()
```



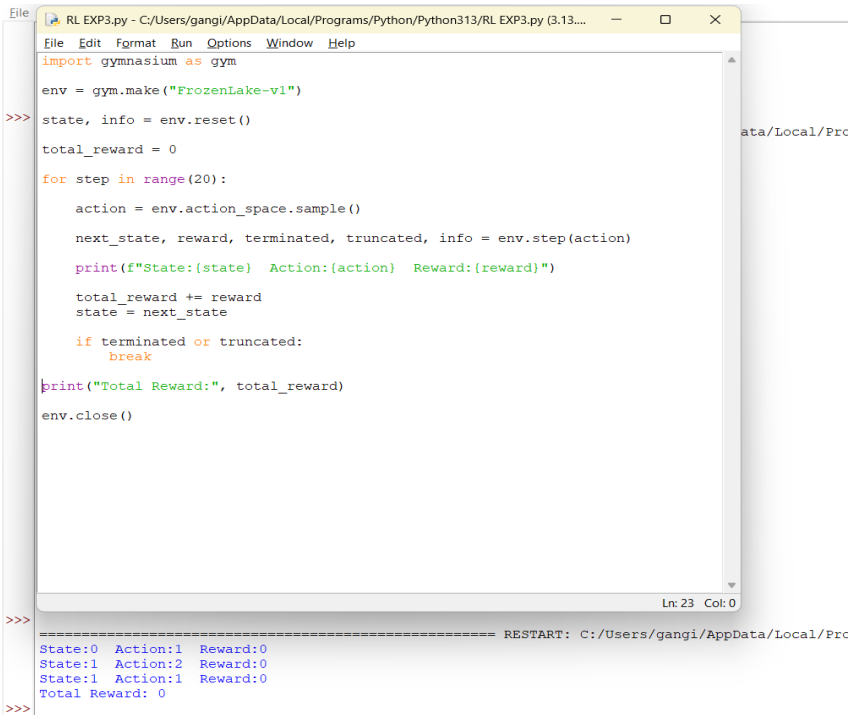
Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

PROGRAM:

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

total_reward = 0

for step in range(20):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print(f"State:{state}  Action:{action}  Reward:{reward}")

    total_reward += reward

    state = next_state

    if terminated or truncated:

        break

print("Total Reward:", total_reward)

env.close()
```

OUTPUT



Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems

PROGRAM:

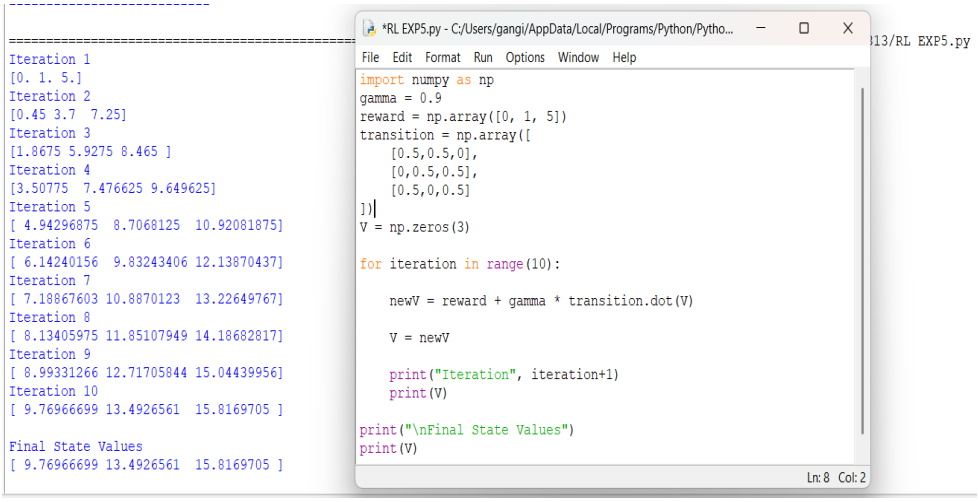
```
import numpy as np

states = ["A", "B", "C"]

transition = {
    "A": {"left": "B", "right": "C"},
    "B": {"left": "A", "right": "C"},
    "C": {"left": "A", "right": "B"}
}

reward = {
    "A": 1,
    "B": 2,
    "C": 5
}

state = "A"

print("Initial State:", state)

for i in range(10):
    action = np.random.choice(["left", "right"])
    next_state = transition[state][action]
    print(f"Step {i+1}")
    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward[next_state])
    print("----------------------------")
    state = next_state
```

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

PROGRAM:

```
import numpy as np
gamma = 0.9
reward = np.array([0, 1, 5])
transition = np.array([
    [0.5,0.5,0],
    [0,0.5,0.5],
    [0.5,0,0.5]
])
V = np.zeros(3)
for iteration in range(10):
    newV = reward + gamma * transition.dot(V)
    V = newV
    print("Iteration", iteration+1)
    print(V)
print("\nFinal State Values")
print(V)
```



Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ε-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

PROGRAM:

) import random

epsilon = 0.2

Q = [10, 15, 8]

for episode in range(20):

if random.random() < epsilon:

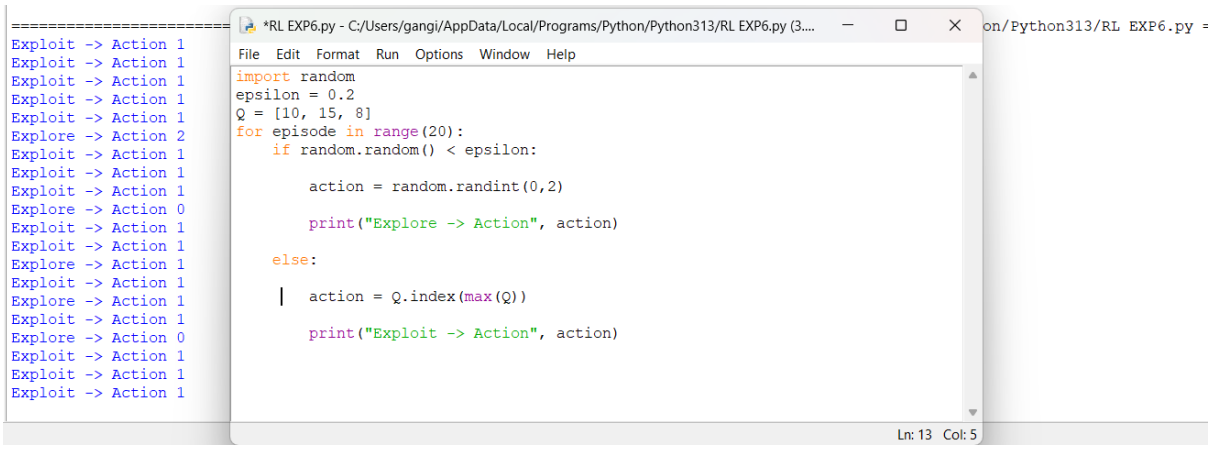
action = random.randint(0,2)

print("Explore -> Action", action)

else:

action = Q.index(max(Q))

print("Exploit -> Action", action

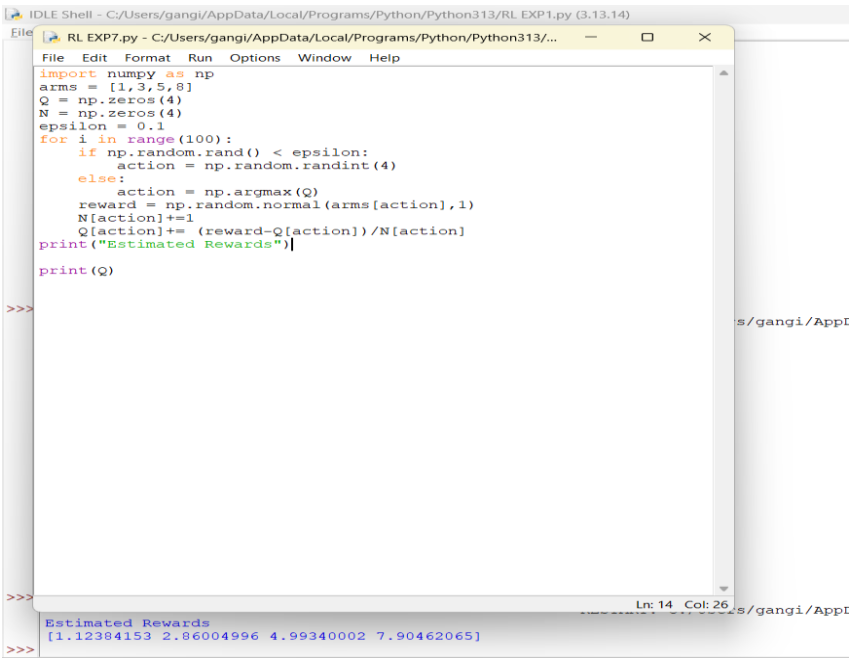


Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ε-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods

PROGRAM:

```
import numpy as np

arms = [1,3,5,8]

Q = np.zeros(4)

N = np.zeros(4)

epsilon = 0.1

for i in range(100):

    if np.random.rand() < epsilon:

        action = np.random.randint(4)

    else:

        action = np.argmax(Q)

    reward = np.random.normal(arms[action],1)

    N[action]+=1

    Q[action]+= (reward-Q[action])/N[action]

print("Estimated Rewards")

print(Q)
```



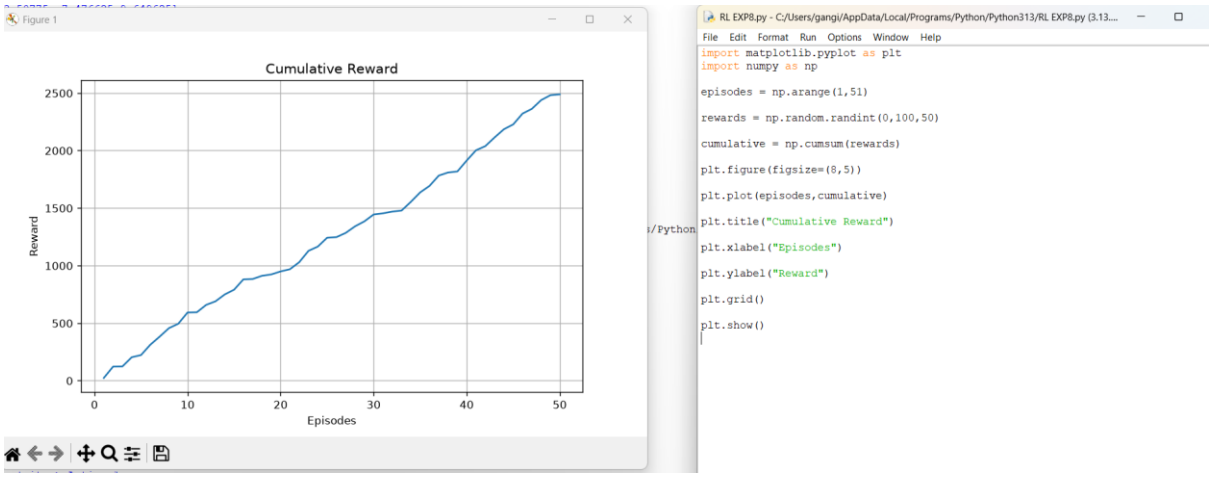
Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

PROGRAM:

```
import matplotlib.pyplot as plt

import numpy as np

episodes = np.arange(1,51)

rewards = np.random.randint(0,100,50)

cumulative = np.cumsum(rewards)

plt.figure(figsize=(8,5))

plt.plot(episodes,cumulative)

plt.title("Cumulative Reward")

plt.xlabel("Episodes")

plt.ylabel("Reward")

plt.grid()

plt.show()
```

OUTPUT



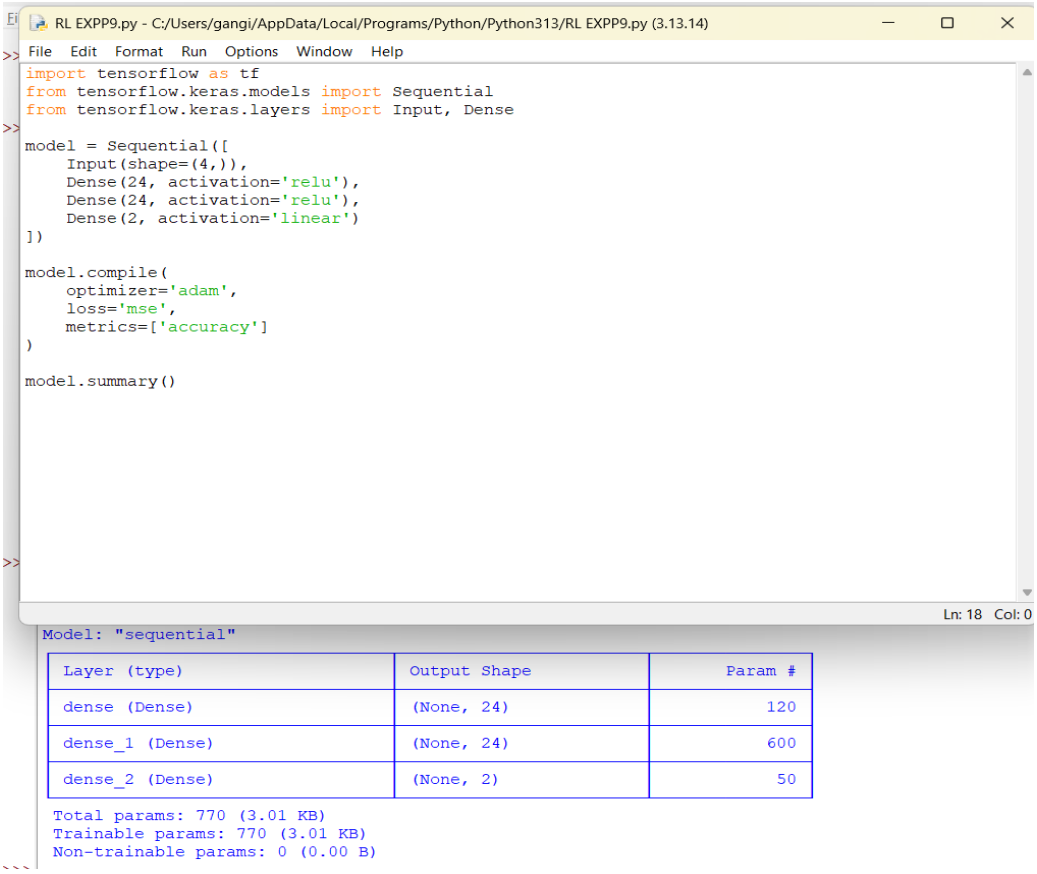
Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments

PROGRAM:

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(24,input_shape=(4,),activation='relu'))

model.add(Dense(24,activation='relu'))

model.add(Dense(2,activation='linear'))

model.compile(optimizer='adam',

        loss='mse',

        metrics=['accuracy'])

model.summary()
```

OUTPUT



Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

PROGRAM:

```
import gymnasium as gym
import numpy as np
env = gym.make("CartPole-v1")
episodes = 10
for episode in range(episodes):
    state, info = env.reset()
    total_reward = 0
    done = False
    while not done:
        action = env.action_space.sample()
        state,reward,terminated,truncated,info = env.step(action)
        total_reward += reward
        done = terminated or truncated
    print("Episode:",episode+1,
          " Reward:",total_reward)
env.close()
```